HTTP Low and Slow DoS Attack Detection using LSTM based deep learning

Bronjon Gogoi
Centre of Excellence for Application Security
Assam State Centre
National Informatics Centre
Guwahati,Assam, India
asm-bronjon@nic.in

Tasiruddin Ahmed
Centre of Excellence for Application Security
Assam State Centre
National Informatics Centre
Guwahati,Assam, India
asm-tasir@nic.in

2022 IEEE 19th India Council International Conference (INDICON) | 978-1-6654-7350-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/INDICON56171.2022.10039772

Abstract— **HTTP low and slow DoS is a kind of attack wherein an attacker sends a stream of very slow HTTP requests to a web server. These slow HTTP requests are aimed at specific applications or server resources. By sending HTTP requests very slowly, these attacks waste the server's resources as the server has to wait for the slow HTTP requests to be completed. HTTP low and slow DoS attacks are primarily used against thread-based web servers like Apache and IIS. By sending very slow requests, these attacks tie up the server threads. This results in a Denial of service for other legitimate users. Since low and slow DoS attacks do not require flooding or sending a large number of HTTP requests, they are not easily detected. Traditional network layer tools cannot detect low and slow DoS attacks and other mitigation and detection strategies are required for the detection of low and slow DoS attacks. In this paper, we propose an LSTM deep learning-based approach for detecting HTTP DoS attacks. The proposed study was conducted on the CIC DoS dataset and a synthetically generated dataset and achieved an impressive accuracy of 0.99.**

Keywords— Low and Slow DoS attacks, LSTM, Deep learning, cyber security

I. INTRODUCTION

HTTP low and slow DoS is a kind of resource exhaustion attack, wherein an attacker tries to exhaust the resources of a thread-based webserver by tying up the threads by sending very slow HTTP requests. These kinds of attacks do not need to generate a large number of requests and also do not require a huge amount of bandwidth [1]. Since these attacks do not require significant resources, they can be launched easily with easily available tools. These attacks do not generate a significant amount of traffic and therefore are not easily detected and can even go unnoticed for a long period of time. The traditional method of detecting DoS and DDoS attacks at the network layer is not effective against HTTP low and slow DoS attacks because the traffic at the network layer looks like legitimate traffic due to the low volume traffic. The most immediate impact of an HTTP low and slow DoS attack is that legitimate users are denied access to the server, resulting in a denial-of-service scenario. Moreover, this also results in revenue loss and loss of user trust in the web application. Many tools can be used for launching a low and a slow attack and an attacker can easily take down an unprotected server. Slow loris and R.U.D.Y (R U Dead Yet) are two popular slow DoS tools available freely and easily accessible over the internet and can be used by anyone with a personal computer.

Though there are techniques for mitigation of slow DoS attacks like WAFs (Web Application Firewalls) and CDN (Content Distribution Networks), and server configuration techniques, they cannot be easily implemented mainly due to costs and lack of application security expertise and difficulty in implementing complex configurations and rules. Moreover, the existing techniques cannot prevent every low and slow DoS attack. Due to this, detection of low and slow DoS is an important part of cyber security for any organization hosting a webserver. Detection of low and slow DoS attacks after the incident can allow the administrators to take remedial measures like blocking the IP address of the attacker and other preventive measures.

Slow and low attacks can be detected through web server log analysis as well through analysis of network traffic. HTTP traffic can be captured using packet capturing tools and can be later analyzed for any traces of low and slow DoS. Web servers maintain logs of HTTP requests which can be analyzed for traces of low and slow DoS attacks. This paper presents an LSTM based deep learning approach for the detection of low and slow DoS attacks through analysis of HTTP traffic captured using packet capture tools.

The main contributions of this paper are as follows-

- An algorithm for parsing PCAP files for isolating HTTP slow and low DoS requests from normal HTTP requests.
- Generation of synthetic data for training LSTM based deep learning algorithms for classifying HTTP flooding requests and benign HTTP requests.
- A method for classification of HTTP flooding requests and benign HTTP requests through sequence classification using LSTM based deep learning.

The remainder of the paper is organized as follows. Section II presents the background, section III presents related work, section IV presents the proposed approach and finally, the conclusion is presented in section V.

II. BACKGROUND

This section provides a background on various methods of HTTP low and slow DoS attacks. Low and slow DoS attacks work by sending HTTP requests very slowly and thereby tying up the threads of a thread-based web server like Apache web server, IIS (Internet Information Services). The three kinds of slow DoS attacks that are commonly used namely slowloris, slow post [2], and slow read [3] are discussed next.

978-1-6654-7350-7/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on August 22,2026 at 16:13:11 UTC from IEEE Xplore. Restrictions apply.

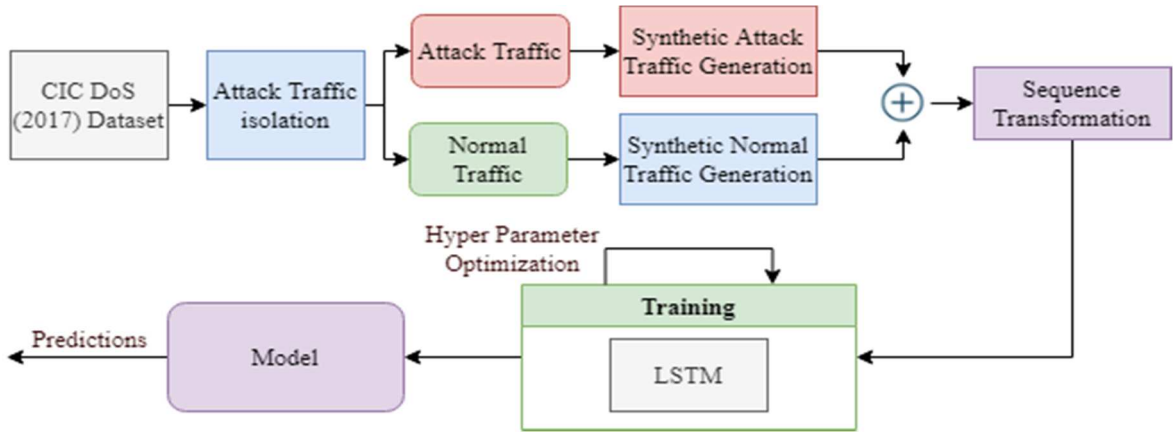


Fig. 1. High level Architecture of the proposed system

A. *Slow Header or Slow Loris*

The slow header or slow loris attack works by sending incomplete HTTP requests. The server waits for the HTTP requests to complete thereby wasting its resources and when all resources are exhausted, other legitimate requests are denied service. The incomplete requests are made by not sending a CRLF (Carriage Return + Line Feed) sequence marking the end of an HTTP request.

B. *Slow Body or Slow Post*

The slow body or rudy works by sending post data very slowly to the server. In this attack, the attacker sends a post request to the server and follows by sending data very slowly, usually 1 byte at a time thereby keeping the server threads busy servicing the slow post requests and denying service to legitimate requests.

C. *Slow Read*

The slow read attack works by sending a legitimate HTTP request but reads the response very slowly, usually 1 byte at a time. As result, the server has to keep the connection open so that the client can complete reading the data. This keeps the server threads busy, serving slow read requests and when the server's resources are exhausted, requests from legitimate clients are denied, leading to a DoS.

III. Related Work

Multiple approaches have been proposed for the detection of Slow DoS attacks with the most common being signature and rule-based systems which are not effective. This section discusses the newer anomaly-based, machine learning, and deep learning-based approaches.

Hossein Hadian Jazi et al proposed an approach to detecting low and slow DoS attacks using the CSUM algorithm [4]. An anomaly-based approach using Fourier transformation was proposed in [5] by M. Aiello, E. Cambiaso, M. Mongelli, and G. Papaleo. N. Tripathi, N. Hubballi, and Y. Singh proposed a method that compares the probability distribution of low and slow DoS requests and normal requests using Hellinger distance in [6]. In [7], S. Tayama and H. Tanaka proposed a method based on the analysis of Maximum Connections (MC) and Server Limit (SL) parameters of an Apache web server. C. G. Chaves, S. P. Azad, J. Sepulveda, and T. Hollstein proposed a Finite State Machine (FSM) based approach that can be implemented in hardware in [8]. M. Sikora, R. Fujdiak, and J. Misurec proposed a signature-based approach in [9] wherein a set of network communication thresholds are defined for attack detection. A. Reed, L. S. Dooley, and S. K. Mostefaoui proposed a detection approach based on the analysis of packet lengths and packet delta times in [10]. N. Tripathi and N. Hubballi proposed an anomaly-based approach to detecting low and slow DoS attacks against HTTP/2.0 using Chi-Square test in [11].

Recently several machine learning and deep learning-based approaches have been proposed for the detection of low and slow DoS attacks. Kemp, CliffordCalvert, Chad Khoshgoftaar, Taghi M. proposed a shallow machine learning based approach and compared the eight different classifiers but the dataset used is not publicly available in [12]. N. Muraleedharan and B. Janet proposed a deep learning-based approach in [13] using a vanilla ANN. In [14], J. A. Perez-Diaz, I. A. Valdovinos, K. K. R. Choo, and D. Zhu used shallow machine learning algorithms for the classification of traffic and achieved an accuracy of 0.95. V. de M. Rios, P. R. M. Inácio, D. Magoni, and M. M. Freire used Fuzzy logic and shallow machine learning algorithms in [15] and achieved a high accuracy but the test set is small so further study is required for testing accuracy on a larger test set.

IV. Proposed Approach

The architecture of the proposed approach is shown in Fig. 1. The CIC DoS (2017) dataset is merged with a synthetic dataset that has been generated to mimic the behavior of attacks. Following the merge, the attack requests are isolated from the normal dataset. The data is then labeled as either attack or benign. The labeled data is then transformed into sequences for feeding to the LSTM based sequence classification module. The model obtained by training the LSTM module is then used for making predictions. The algorithm for isolation of attacks from normal traffic, sequence transformation, and LSTM module is discussed in subsequent sections.

A. *Dataset Description*

The proposed approach uses the CIC DoS (2017) dataset for training and evaluation of the LSTM network. The CIC DoS dataset is a public dataset containing slow HTTP DoS attack traces and normal HTTP traffic. The packet capture file of the dataset contains traffic capture of attack and normal

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on August 22,2026 at 16:13:11 UTC from IEEE Xplore. Restrictions apply.

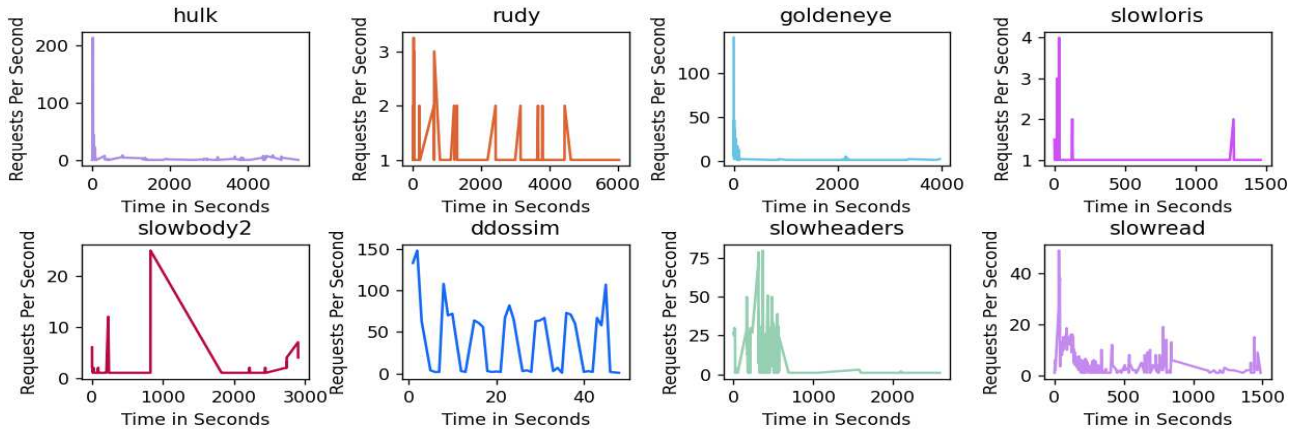


Fig. 2. Requests per second metric vs Time in Seconds of various DoS attacks

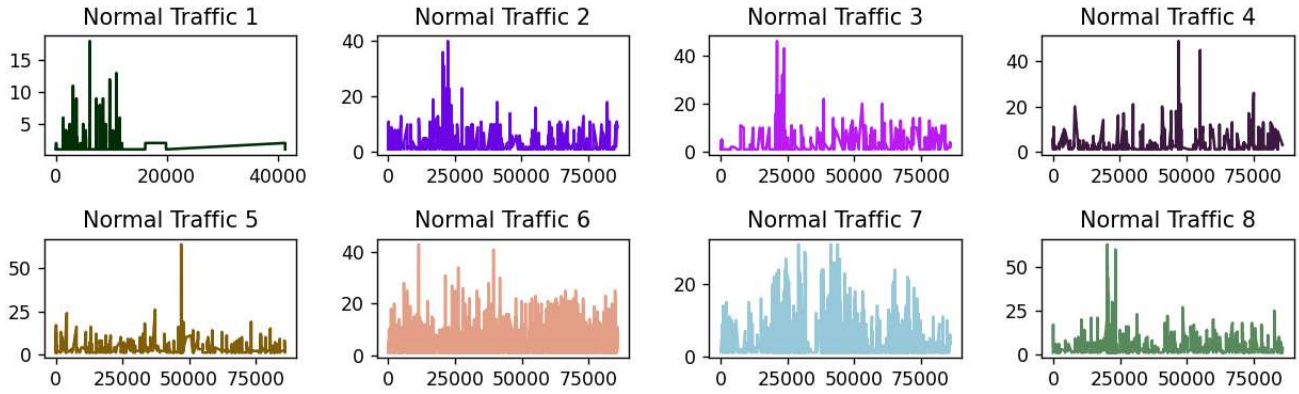


Fig. 3. Requests per second vs Time in Seconds of normal traffic from different hosts

HTTP traffic of 24-hour duration and the size of the captured traffic is 4.6 GB.

The attack traces contained in the dataset is representative of the most common type of slow DoS attacks and are generated using common tools available for performing HTTP DoS attacks. The tools used for generating the attacks along with the number of requests per second generated by the tool are shown in Fig. 2 and the traffic generated by normal HTTP traffic is shown in Fig. 3. From Fig. 2, from the graph of traffic generated by DoS tools, it is seen that the attacks last for a small duration of time and there is a high variation in the request rate. In Fig. 3, normal HTTP requests coming from 8 different hosts are shown. The normal traffic lasts for a longer duration compared to the attack traffic and the variation in the request rate is more or less uniform. This difference in behavior in terms of the number of requests per second and duration of traffic in terms of the number of seconds between attack and normal HTTP traffic can be used for detecting low and slow DoS attacks. The various statistics of the dataset are shown in Table 1.

TABLE 1. DATASET STATISTICS

Metric	Value
Total number of HTTP requests	261026
Number of slow DoS HTTP requests	32318
Number of benign HTTP request	228708
Traffic capture duration	24 Hours
Volume (Size in Gb)	4.6
Number of DoS Tools used	8

B. Attack Traffic isolation

The proposed approach aims to separate low and slow DoS HTTP traffic from normal HTTP traffic. For training the classifier, the dataset is required to be split into attack traffic and normal traffic. Algorithm 1 is used for the isolation of attack traffic from normal benign traffic. The algorithm takes as input a list of time stamps at which the attack occurred, a list of IP addresses at which the attacks were directed, and outputs a dictionary of IP addresses where each key is the IP address of the attacker and the values are HTTP requests part of a slow DoS attack coming from that IP address. All other HTTP requests that are not part of the attack requests are considered normal benign HTTP traffic. The timestamps and the hosts at which the attack requests are directed are included in the CIC DoS (2017) dataset which makes it easier to isolate the attack traffic from the normal traffic.

C. Synthetic Data Generation

The number of attack traces in the CIC DoS dataset is eight. To further increase the data for training the LSTM network used for classification, synthetic attack, and normal traffic data are generated. The attack synthetic data is generated to mimic the distribution of the attack requests pattern found in the CIC dataset and the normal synthetic data mimics the distribution of the normal requests in the CIC

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on August 22,2026 at 16:13:11 UTC from IEEE Xplore. Restrictions apply.

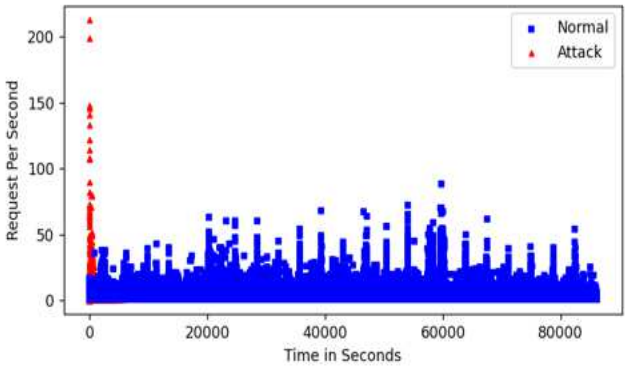


Fig. 4. RPS vs Time of requests in CIC DoS (2017) Dataset

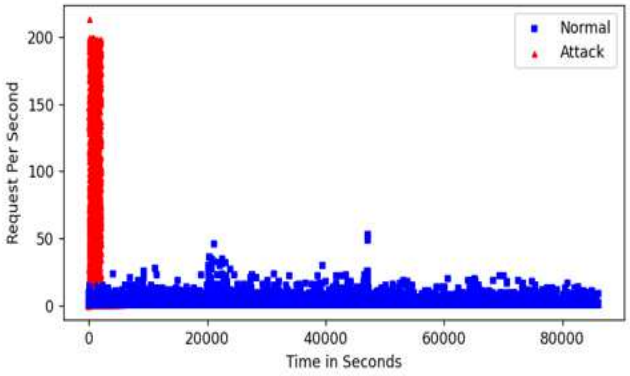


Fig. 5. RPS vs Time of Generated Synthetic Traffic

Algorithm 1: Isolate attack traces from normal Traffic

Input:

List of attack timestamps

List of attack IP address

List of requests

Output:

Dictionary of attack HTTP requests keyed by IP address of the attacker

```
time_stamps ← list (attack time stamps)
attack_ips ← list (attack IP addresses)
list_requests ← list (all_requests)
list_attack_requests ← [ ]
ip_attack_dictionary = dictionary ()
for index in range (0, length (time_stamps):
    for each request in list_requests:
        if time_stamp(request) >= time_stamps[index] and
        time_stamp(request) < time_stamp[index+1]
        list_attack_requests.add(request)
unique_attack_ips = unique_IP (list_attack_requests)
for unique_ip in unique_attack_ips:
    for request in list_requests:
        if request[ip] == unique_ip:
        ip_attack_dictionary[unique_ip].add(request)
return ip_attack_dictionary
```

dataset. The synthetic data generation process generates 10000 attack traces and 10000 normal traces. The scatter plot of the CIC DoS dataset and the scatter plot of the generated data are shown in Fig. 4 and Fig. 5 respectively. The scatter plots in Fig. 4 and 5 plot the request per second of a given attack trace against the time in second and each attack trace correspond to one unique IP address. From Fig. 4 and 5, it can be seen that the request pattern of the synthetically generated traffic is similar to the traffic obtained from the CIC dataset. Both the CIC and the synthetic dataset contain attack samples that last for a duration significantly less than the normal traffic duration. The synthetic data is merged with the CIC dataset to obtain the final dataset that is used for training and evaluation of the LSTM based model.

D. Sequence Transformation

LSTM network requires input in the form of a sequence where the sequence is three dimensional. The sequence consists of samples, timesteps, and features, where each sample consists of several timesteps and each timestep contains several features. The timesteps should be of equal length and the number of features should be also the same for each timestep. The number of samples in the sequence can be 1 or more. To classify the HTTP requests as either low and slow DoS attacks or benign, they are transformed into sequences. The sequence can be mathematically represented as follows –

$$Sequence = (S_i)_{i=1}^{10000}$$

$$S_i = \{(t_j, r_j\}, \; 0 \leq j \leq 180, 0 \leq t \leq 84000 \; and \; r \in Z^+$$

The 10000 attack traces correspond to traffic coming from 10000 different IP addresses and these traces were obtained from the CIC DoS dataset and the synthetically generated traffic. Each attack trace originating at a particular IP address becomes one sample S_i. Each sample consists of t_j and r_j where t is time in second and r is request per second at time t coming from the source IP address of the attack. Each sample can contain 180 randomly chosen time steps and since the duration of traffic capture is 24 hours, t can have a maximum value of 84000 and a minimum value of 0. The request per second metric can be any positive integer value. An example sequence is shown below with 2 timesteps is shown below

Sequence = [[0,100], [1,200]]

The above sequence contains 2 samples and each sample contain a pair of number where the first number is the time t and the second number is r, which is the request per second metric coming from the source IP address of a particular attack trace. In the above example, at the 0th timestamp, there were 100 requests and at the 1st second, there were 200 requests. The sequence of samples, timesteps, and features are input to the LSTM network for classification as either low and slow DoS attacks or benign HTTP requests.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on August 22,2026 at 16:13:11 UTC from IEEE Xplore. Restrictions apply.

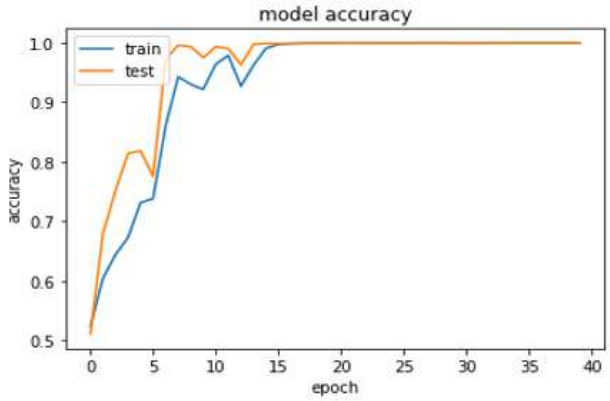


Fig. 6. Model Accuracy Graph

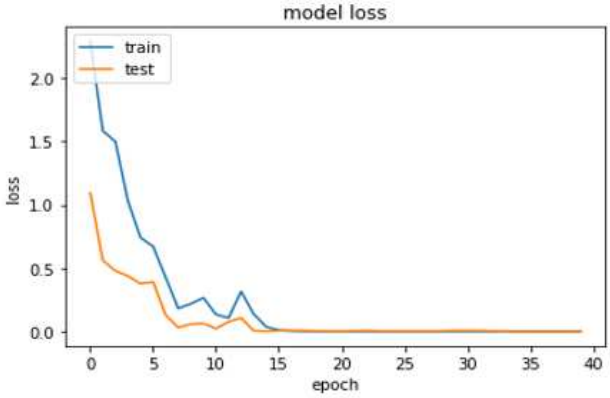


Fig. 7. Model Loss Graph

Layer (type)	Output Shape
masking (Masking)	(None, 180, 2)
lstm (LSTM)	(None, 180, 64)
dropout (Dropout)	(None, 180, 64)
lstm_1 (LSTM)	(None, 32)
dropout_1 (Dropout)	(None, 32)
dense (Dense)	(None, 8)
dropout_2 (Dropout)	(None, 8)
dense_1 (Dense)	(None, 1)

Fig. 8. LSTM network layers

TABLE 2. HYPERPARAMETERS OF LSTM NETWORK

Hyperparameter	Value
Optimizer	Adam
Loss Function	Binary Cross Entropy
Activation Function	Relu
Number of Epochs	40
Batch Size	512
Number of Hidden Layers	3
Learning Rate	0.01
Dropout	0.3
Weight Initialization	Uniform

E. LSTM Training

LSTMs are widely used for sequence classification since LTSMs do not have the exploding gradient and vanishing gradient problems that are inherent to the vanilla RNN (Recurrent neural network) networks. LSTM solves the problem of exploding gradient and vanishing gradient through the use of constant error back propagation. LSTMs can remember long sequences and are suited for most sequence classification tasks. In the proposed approach LSTM networks are used for the classification of sequences of requests as either benign HTTP requests or as low and slow HTTP DoS attack requests.

The sequences obtained in the previous step discussed in section IV(C) are used for training the LSTM network. The dataset is split into two sets – the training set and the test set. The training set contains around 7000 samples and the test set contains around 3000 samples. This makes the split ratio 70:30. The LSTM network is trained using the training set and the test set is used for testing the performance of the trained classifier. The training set which is a sequence of samples is input to the LSTM network. The task of the LSTM network is to classify each sample of requests as either attack or benign. The architecture of the LSTM network used in the proposed approach is shown in Fig. 8. The first layer of the network is a masking layer that masks any zero values in the input. The second layer is an LSTM layer with 64 LSTM units. The shape of the input to the layer is (180,64) which means that the length of each sample is 180. The first LSTM layer is followed by a dropout regularization layer with a dropout value of 0.3. Dropout regularization is used to prevent overfitting. The regularization layer is followed by a second LSTM layer with 32 units. This layer is again followed by a regularization layer with a dropout value of 0.3. The penultimate layer is a Dense layer with 8 units. The final layer is a Dense layer with 1 unit. The final layer uses a sigmoid function as an activation function while the hidden layers use relu activations. The output of the final layer is a probability value in the range of 0 to 1.0. The probability value obtained from the last Dense layer is used to predict whether a sequence of requests is a low and slow DoS attack or benign. For a given sequence if the probability value obtained is less than 0.5, it is classified as a benign sequence of requests and if the probability value is greater than equal to 0.5, it is considered an attack. The hyperparameters used in the training process are shown in Table 2.

The model accuracy during training is shown in Fig. 6 and model loss is shown in Fig. 7. Fig 6. shows the accuracy in terms of the training set and the test set. Both the training accuracy and test accuracy increase with the increase in the number of epochs. A huge difference between the train and test accuracy indicates the model is either underfitting or overfitting the data, but in the graph in Fig. 6, it can be seen that after about 15 epochs, both the train and test accuracy

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on August 22,2026 at 16:13:11 UTC from IEEE Xplore. Restrictions apply.

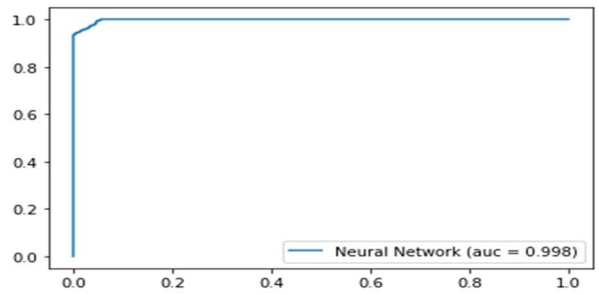


Fig. 8. AUC-ROC curve

reaches 0.99 and both the curve converges. This indicates the model is not underfitting or overfitting, and the model can generalize well. The model loss graph is shown in Fig. 7. Similar to the model accuracy curve, the model loss for the training set and the test set decreases with the increase in the number of epochs. The training loss curve and the test curve converge after around 15 epochs indicating that the model is neither underfitting nor underfitting. This indicates that the classifier can classify the samples with decent accuracy.

The performance of the trained model is evaluated using the AUC-ROC curve. AUC-ROC curve measures the degree of separability between the classes. A model with a high AUC indicates a higher ability on the part of the model to distinguish between a positive class and a negative class. The AUC-ROC curve of the trained model is shown in Fig. 8 where the TPR (True Positive Rate) is plotted along the y-axis and the FPR (False Positive Rate) is plotted along the x-axis. The AUC achieved by the proposed model is around 0.99 which indicates that the model is capable of separating the positive and negative classes quite accurately and there is no significant overfitting and underfitting.

V. Conclusion

Slow DoS is one of the most commonly used resource exhaustion attacks launched against web servers by malicious users and cyber criminals. This is due to the ineffectiveness of traditional intrusion detection and prevention tools against Slow DoS attacks. In this paper, an LSTM based deep learning method is proposed for the detection of Slow DoS attacks. The proposed approach transforms requests coming from malicious users into sequences and classifies them as either DoS attack traffic or benign traffic. The experiments were conducted on the well-known dataset CIC DoS (2017) and also on synthetically generated data. The proposed approach achieved a high accuracy of 0.99. The false positive and false negative rates were also quite low in the range of 0.01. The experiments conducted and the results demonstrate that LSTM based deep learning approach can be used for the detection of Slow DoS attacks with a high degree of accuracy.

References

[1] "What is a low and slow attack? low and slow ddos attack ... - cloudflare." [Online]. Available: https://www.cloudflare.com/learning/ddos/ddos-low-and-slow-attack/. [Accessed: 13-Jun-2022]

[2] E. Cambiaso, G. Papaleo, and M. Aiello, "Taxonomy of slow DoS attacks to web applications," *Commun. Comput. Inf. Sci.*, vol. 335 CCIS, no. i, pp. 195–204, 2012, doi: 10.1007/978-3-642-34135-9_20.

[3] S. Tayama and H. Tanaka, "Analysis of slow read DoS attack and communication environment," *Lect. Notes Electr. Eng.*, vol. 425, pp. 350–359, 2018, doi: 10.1007/978-981-10-5281-1_38.

[4] Hossein Hadian Jazi, Hugo Gonzalez, Natalia Stakhanova, and Ali A. Ghorbani. "Detecting HTTP-based Application Layer DoS attacks on Web Servers in the presence of sampling." Computer Networks, 2017

[5] M. Aiello, E. Cambiaso, M. Mongelli, and G. Papaleo, "An on-line intrusion detection approach to identify low-rate DoS attacks," *Proc. - Int. Carnahan Conf. Secur. Technol.*, vol. 2014-October, no. October, 2014, doi: 10.1109/CCST.2014.6987039.

[6] N. Tripathi, N. Hubballi, and Y. Singh, "How Secure are Web Servers? An empirical study of Slow HTTP DoS attacks and detection," *Proc. - 2016 11th Int. Conf. Availability, Reliab. Secur. ARES 2016*, pp. 454–463, 2016, doi: 10.1109/ARES.2016.20.

[7] S. Tayama and H. Tanaka, "Analysis of slow read DoS attack and communication environment," *Lect. Notes Electr. Eng.*, vol. 425, pp. 350–359, 2018, doi: 10.1007/978-981-10-5281-1_38.

[8] C. G. Chaves, S. P. Azad, J. Sepulveda, and T. Hollstein, "Detecting and Mitigating Low-and-Slow DoS Attacks in NoC-based MPSoCs," *Proc. - 2019 14th Int. Symp. Reconfigurable Commun. Syst. ReCoSoC 2019*, pp. 82–89, 2019, doi: 10.1109/ReCoSoC48741.2019.9034934.

[9] M. Sikora, R. Fujdiak, and J. Misurec, "Analysis and detection of application-independent slow Denial of Service cyber attacks," *Proc. - 2021 IEEE Int. Conf. Intell. Secur. Informatics, ISI 2021*, pp. 2–7, 2021, doi: 10.1109/ISI53945.2021.9624789.

[10] A. Reed, L. S. Dooley, and S. K. Mostefaoui, "A Reliable Real-Time Slow DoS Detection Framework for Resource-Constrained IoT Networks," pp. 1–6, 2022, doi: 10.1109/globecom46510.2021.9685612.

[11] N. Tripathi and N. Hubballi, "Slow rate denial of service attacks against HTTP/2 and detection," *Comput. Secur.*, vol. 72, pp. 255–272, 2018, doi: 10.1016/j.cose.2017.09.009.

[12] C. Kemp, C. Calvert, and T. M. Khoshgoftaar, "Detection Methods of Slow Read DoS Using Full Packet Capture Data," *Proc. - 2020 IEEE 21st Int. Conf. Inf. Reuse Integr. Data Sci. IRI 2020*, pp. 9–16, 2020, doi: 10.1109/IRI49571.2020.00010.

[13] N. Muraleedharan and B. Janet, "A deep learning based HTTP slow DoS classification approach using flow data," *ICT Express*, vol. 7, no. 2, pp. 210–214, 2021, doi: 10.1016/j.icte.2020.08.005.

[14] J. A. Perez-Diaz, I. A. Valdovinos, K. K. R. Choo, and D. Zhu, "A Flexible SDN-Based Architecture for Identifying and Mitigating Low-Rate DDoS Attacks Using Machine Learning," *IEEE Access*, vol. 8, pp. 155859–155872, 2020, doi: 10.1109/ACCESS.2020.3019330.

[15] V. de M. Rios, P. R. M. Inácio, D. Magoni, and M. M. Freire, "Detection of reduction-of-quality DDoS attacks using Fuzzy Logic and machine learning algorithms," *Comput. Networks*, vol. 186, no. December 2020, p. 107792, 2021, doi: 10.1016/j.comnet.2020.107792.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on August 22,2026 at 16:13:11 UTC from IEEE Xplore. Restrictions apply.